

Title: Zombie Survival Prototype

A first-person zombie survival prototype developed in Unity, exploring player movement, enemy AI, navigation, and animation systems.

The project focuses on implementing responsive character controls and a simple AI system that allows enemies to detect, pursue, and attack the player based on distance and field of view.

Through this prototype, I explored the practical integration of C# scripting, CharacterController, NavMesh navigation, and Animator Controller systems to create a functional gameplay experience.

**Created by:-
Ramanjeet Arneja**

Table of Contents

1. Game Overview	2
2. Gameplay Mechanics	2
2.1 Player Movement	2
2.2 Camera Control	2
2.3 Gravity and Jumping	2
2.4 Crouch System	2
3. Enemy (Zombie) Behavior	2
3.1 Idle State	2
3.2 Walking State	2
3.3 Attack State	2
3.4 Animation System	2
4. NavMesh Integration	3
5. Objective and Rules	3
6. Controls	3
7. Characters	3
7.1 Player	3
7.2 Zombie Enemy	5
8. Zombie Animator Controller	6
8.1 States	7
8.2 Parameters	7
8.3 Transitions	7
8.4 Entry and Exit	7
9. Environment / Level Design	8
10. Technical Details	8
10.1 Game Engine	8
10.2 Programming Language	8
10.3 Main Scripts	8
10.4 Components Used	8
11. Reflection	8

Game Overview

The project is a small first-person game in which the player plays a game as a zombie and the goal is to survive. Character control and simple enemy AI behaviour is the central element of the game. The environment is open to an exploration which can be controlled by using keyboard and mouse, it is possible to avoid or face zombies which respond to the distance and angle. The game illustrates fundamental mechanics of the game such as movement, animation management, camera control of the player, and logic of the enemy.

Gameplay Mechanics

- **Player Movement:**
The gamer is able to walk, run, jump, and crouch. Running increases speed of the character whereas crouching lowers speed and height. The camera also tracks the head of the player thus being able to look anywhere with the mouse.
 - **Camera Control:**
The movement of the mouse will determine how the player looks. The vertical rotation is restricted to allow the player to look as far as backward.
 - **Gravity and Jumping:**
When the player is not on the ground, gravity acts on him, and the jump button provides an upward movement to make the player jump.
 - **Crouch System:**
When the player presses the crouch key, he/she becomes shorter and slower, which can be used to act as a stealth player.
-

Enemy (Zombie) Behavior

- **Idle State:**
In fact, when the player is far or not in the visual range of the zombie, the zombie remains idle.
 - **Walking State:**
On the player being spotted within range (an approximation of 50 units) and within the 45deg viewing cone, the zombie turns towards the player and starts walking towards it.
 - **Attack State:**
The zombie begins the attack animation when the player is so close (less than 3 units).
 - **Animation System:**
The animations of the enemy are regulated with the help of the parameter of a switch in the Animator component, which is determined by the calculation of the distance and angles.
-

NavMesh Integration

This project uses the NavMesh system of Unity to enhance the movement and pathfinding of the enemy. The NavMesh enables the zombie AI to move more intelligently around the obstacles and the player. This assists in avoiding trapping along with straight walking by zombies making them more authentic to their actions.

The baking of the ground in the scene is indicating that it can be walked on. The zombie GameObjects have the NavMeshAgent component that is used to go around the mesh, automatically seeking a path around walls and objects. Enemy logic Enemy logic is no more complicated than NavMeshAgent navigation combined with personalized detection code, meaning the zombies do not start chasing the player until they detect them.

This design allows zombie motion to be more fluid and the quality of the gameplay to be improved since it enables the AI to be more navigable through the surroundings.

Objective and Rules

This prototype aims to explore and survive as long as could be achieved without coming too close to the zombie. No win or lose-conditions exist but the key idea of this version is to show functioning player controls, as well as enemy logic.

Controls

Action	Key / Input
Move Forward / Backward	W / S
Move Left / Right	A / D
Run	Left Shift
Jump	Space
Crouch	R
Look Around	Mouse Movement

Characters

Player

The central figure is The Player and he is played in the first person point of view. With help of keyboard and mouse controls, the Player may walk, run, jump, crouch and look around. The movement is fluid and physics based and enables the realistic exploration of the environment. It has the camera on the head of the player so that it can provide an immersive mouse-look both during vertical and horizontal movement. Crouching will slow down the player and bring him closer to the ground, whereas running will accelerate him. There is the gravity and jumping mechanism to emulate actual movement. Skills of the Player are handled by a C# script named as PlayerMovement, which takes care of the movement, rotation, gravity, camera, and crouch state.

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;

// GameObject has a CharacterController component.
[RequireComponent(typeof(CharacterController))]

// The main PlayerMovement class that controls how the player moves and looks around.
public class PlayerMovement : MonoBehaviour
{
    // This is the camera we'll use for looking up and down.
    public Camera playerCamera;

    // Movement speeds for walking and running.
    public float walkSpeed = 6f;
    public float runSpeed = 12f;

    // Jump strength and gravity values.
    public float jumpPower = 7f;
    public float gravity = 10f;

    // How sensitive the mouse movement is and how far up/down we can look.
    public float lookSpeed = 2f;
    public float lookXLimit = 45f;

    // Height values used for standing and crouching.
    public float defaultHeight = 2f;
    public float crouchHeight = 1f;

    // The movement speed when crouching.
    public float crouchSpeed = 3f;

    // This variable stores the current movement direction (x, y, z).
    private Vector3 moveDirection = Vector3.zero;

    // This stores how much the player has looked up or down.
    private float rotationX = 0;

    // We store a reference to the CharacterController here so we can use it easily later.
    private CharacterController characterController;

    // This flag is used to check if the player can currently move or not.
    private bool canMove = true;

    // The Update function runs once per frame.
    void Update()
    {
        // These two lines get the player's forward and right directions based on rotation.
        // That way, movement is always relative to where the player is facing.
        Vector3 forward = transform.TransformDirection(Vector3.forward);
        Vector3 right = transform.TransformDirection(Vector3.right);

        // This checks if the player is holding Left Shift to run.
        bool isRunning = Input.GetKey(KeyCode.LeftShift);

        // We calculate how fast the player should move forward/backward and left/right.
        // If canMove is false, the speed is set to 0.
        float curSpeedX = canMove ? (isRunning ? runSpeed : walkSpeed) * Input.GetAxis("Vertical") : 0;
        float curSpeedY = canMove ? (isRunning ? runSpeed : walkSpeed) * Input.GetAxis("Horizontal") : 0;

        // Save the current vertical movement (used for jumping and gravity) before we overwrite moveDirection.
        float movementDirectionY = moveDirection.y;

        // Combine forward and sideways movement into one movement vector.
        moveDirection = (forward * curSpeedX) + (right * curSpeedY);

        // If the player presses the Jump button and is on the ground, apply upward velocity to jump.
        if (Input.GetButton("Jump") && canMove && characterController.isGrounded)
        {
            moveDirection.y = jumpPower;
        }
        else
        {
            // Otherwise, keep the previous Y velocity (so gravity continues to act).
            moveDirection.y = movementDirectionY;
        }

        // When the player is not grounded, apply gravity every frame to make them fall.
        if (!characterController.isGrounded)
        {
            moveDirection.y -= gravity * Time.deltaTime;
        }

        // This section handles crouching when the R key is pressed.
        // The player becomes shorter and moves slower while crouched.
        if (Input.GetKey(KeyCode.R) && canMove)
        {
            characterController.height = crouchHeight;
            walkSpeed = crouchSpeed;
            runSpeed = crouchSpeed;
        }
    }
}

```

```

else
{
    // When the R key is released, reset height and speeds to normal.
    characterController.height = defaultHeight;
    walkSpeed = 6f;
    runSpeed = 12f;
}

// Finally, we actually move the player based on the movement direction and time.
// Time.deltaTime keeps the movement smooth no matter what the frame rate is.
characterController.Move(moveDirection * Time.deltaTime);

// This handles looking around with the mouse, but only if canMove is true.
if (canMove)
{
    // Vertical rotation (looking up/down). The negative sign inverts the mouse Y axis.
    rotationX += -Input.GetAxis("Mouse Y") * lookSpeed;

    // Clamp the vertical rotation so the player can't look too far up or down.
    rotationX = Mathf.Clamp(rotationX, -lookXLimit, lookXLimit);

    // Apply the up/down rotation to the camera only.
    playerCamera.transform.localRotation = Quaternion.Euler(rotationX, 0, 0);

    // Apply left/right rotation to the entire player body.
    transform.rotation *= Quaternion.Euler(0, Input.GetAxis("Mouse X") * lookSpeed, 0);
}
}

```

Zombie Enemy:

The Zombie Enemy is an easily controlled AI-based character, which evolves according to the position and visibility of the player. When the player is not close to it or not in its field of view, it stays in the idle state. Once the player enters a detection radius and the zombie is within the 45-degree viewing range the zombie begins moving towards the player and turns face-on to look directly at him or her. When the player nears too much, the zombie will change to an attack state. This act is done by testing the distance and angle of the zombie and the player. The movement and animations of the zombie are updated dynamically using an Animator component, which switches its states to idle, walk and attack.

The following is the C# script to allow this simple AI behavior to the zombie, and control the movement towards the player and animation transition:

```

using UnityEngine;
using UnityEngine;

// This script controls a simple zombie AI that idles, walks toward the player, and attacks when close.
public class Zombie : MonoBehaviour
{
    // The zombie's starting health.
    private int health = 100;

    // We'll cache a reference to the Animator so we can trigger animation states quickly.
    private Animator animator;

    // A reference to the player GameObject set from the Inspector so the zombie knows who to follow.
    [SerializeField] GameObject Player;

    // A cached Transform for the player to avoid repeatedly accessing Player.transform in Update.
    private Transform playerref;

    // Start runs once when the object becomes active and is a good place for initialization.
    void Start()
    {
        // Cache the player's transform for faster access.
        playerref = Player.transform;

        // Get the Animator component attached to this GameObject so we can control animations.
        animator = GetComponent<Animator>();
    }

    // Update runs every frame and is where we check distance/angle to the player and change animation states.
    void Update()
    {
        // Find the straight-line distance from the zombie to the player.
        float Distance = Vector3.Distance(playerref.position, transform.position);

        // Direction vector from zombie to player (used for angle calculations and facing).
        Vector3 direction = playerref.position - transform.position;

        // Angle between the direction to the player and the zombie's forward direction.
        float Angle = Vector3.Angle(direction, this.transform.forward);

        // If the player is far away (more than 50 units) or outside the 45-degree view cone, go idle.
        if (Distance > 50 || Angle > 45)
        {
            animator.SetBool("walk", false);
            animator.SetBool("attack", false);
            animator.SetBool("dead", false);
            animator.SetBool("idle", true);
        }

        // If the player is within 50 units and roughly in front (within 45 degrees), start walking toward them.
        if (Distance < 50 && Angle < 45)
        {
            animator.SetBool("walk", true);
            animator.SetBool("idle", false);
            animator.SetBool("attack", false);
            animator.SetBool("isdead", false);

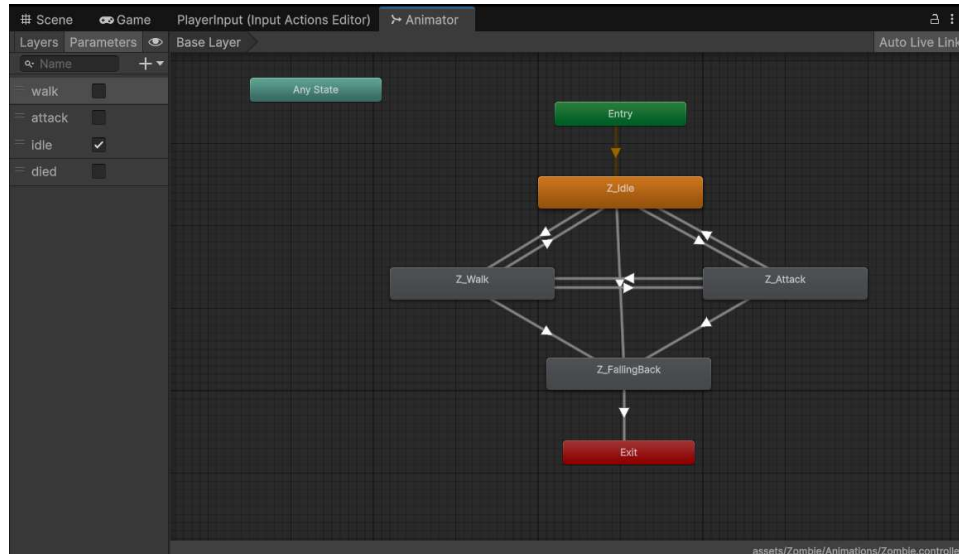
            // Rotate the zombie to face the player horizontally while keeping its own Y position (no tilting).
            this.transform.LookAt(new Vector3(playerref.position.x, this.transform.position.y, playerref.position.z));

            animator.SetBool("Idle", false);
        }

        // If the player gets very close (less than 3 units) and is in front, play the attack animation.
        if (Distance < 3 && Angle < 45)
        {
            animator.SetBool("attack", true);
            animator.SetBool("walk", false);
            animator.SetBool("idle", false);
            animator.SetBool("isdead", false);
        }
    }
}

```

Zombie Animator Controller



In the picture, one can see the Animator Controller of the zombie enemy in Unity. Animator Controller is a mechanism of controlling various animation states and transitions of a character. In the given case, the primary states are: ZIdle, ZWalk, ZAttack, ZFallingBack, and Exit.

States:

- **ZIdle:** The default: The zombie is not doing anything.
- **ZWalk:** This is the condition when the zombie approaches the gamer.
- **ZAttack:** This is triggered upon the zombie being near enough to attack.
- **ZFallingBack:** This is to be used when reacting to attacks like harm or knockback.
- **Exit:** Works to complete or end the flow of animation, e.g. when the zombie dies.

Parameters:

On the left-side, we have such parameters as walk, attack, idle, and died. They are Boolean (true/false) values, which are manipulated by scripts, including the Zombie script, depending on the distance and angle that the zombie has towards the player.

Arrows (Transitions):

The transitions between states are indicated by arrows. As a parameter is pivoted (say attack is set to true) the Animator transitions to the next state. Several arrows imply that the zombie has an opportunity to alternate between idle, walking, and attacking with real-time animation. Other transitions may be back and forth between idle and walk or attack based on the player interaction and AI rules.

Entry and Exit:

- **Entry** The place where animation begins, and it always results in the first state (typically idle).
- **Exit** is where the animation ends which is normally applied when the zombie dies or is deactivated.

This arrangement causes the zombie to act dynamically in-game, state of animation changing according to script logic and parameters. The animation logic of all zombie behaviors is visually arranged by the controller, corresponding to that which is written in the NPC movement section.

Environment / Level Design

It takes place in a minimal open-ground space where it is necessary to test the movement and AI. The design has sufficient space, which enables the player to navigate and move about the space and see how the zombie responds at various distances. The terrain and lighting are also very minimal so as to concentrate on the gameplay, as opposed to the visuals.

Technical Details

- **Game Engine:** Unity
 - **Programming Language:** C#
 - **Main Scripts:**
 - PlayerMovement.cs - handles player movement, camera control, jumping, crouching.
 - Zombie.cs - handles AI logic, animation transitions, and direction facing.
 - **Components Used:** CharacterController, Animator, Camera.
-

Reflection

During the development of this project, I got to know how to move around with the aid of Unity CharacterController and how to integrate it with the camera rotating to create a smooth first-person gameplay.

I also learned basic enemy AI, of which I code checked Vector3 distance and angle checking, and how to manage animations. The initial work with creating and testing animations was a little dull initially, particularly ensuring the transitions were functioning properly, however, this contributed to me being able to have a deeper grasp of how Animator parameters and states respond with scripts.

To make it an even better game, I would have added a health and damage mechanism to both the player and the zombie, sound effects, and perhaps other zombies so that the game can be more dynamic.